

scripts/dc/sql_to_doc.sh — Docs Chain SQL-DDL → System-doc transform

Revision: 1 Last modified: 2026-06-17T00:00:00Z

Overview

scripts/dc/sql_to_doc.sh is the project-owned Docs Chain (§11.4.106) exec: transform named sql-to-doc in `.docs_chain/contexts/sql_definitions.yaml`. It derives the System SQL-reference document (`docs/database/SQL_REFERENCE.md`) from the Core SQL DDL definition files (`core/Database/DDL/**/*.*.sql`).

Authority direction (SAFE DEFAULT, operator-recommended): the `.sql` DDL files are the **sole authority**; the System SQL-reference doc is **derived** from them — a one-way `sql → doc` flow. The reverse projection (`doc → sql`) is **intentionally NOT implemented** because a documentation edit must never silently rewrite authoritative database schema; wiring it requires a §11.4.133 target-System safety review.

Prerequisites

- POSIX sh (the script is `#!/bin/sh`, no bashisms — parses clean under `sh -n`, §11.4.67).
- Base userland: `grep`, `sed`, `sort`, `awk`, `mktemp`.
- The core submodule working tree present so the DDL paths resolve (the transform itself only reads the staged temp inputs Docs Chain provides; the paths matter when Docs Chain stages them).

Usage

The script is **not** normally invoked directly — Docs Chain invokes it per the exec contract. The contract (`CONFIG_SCHEMA.md` §5.2, verified against `docs_chain/internal/runner/runner.go:300–354`):

```
sql_to_doc.sh <in_1> [<in_2> ... <in_N>] <out> [args...]
```

- Docs Chain stages each input node's **content** into a temp file and passes the temp paths in **deterministic sorted-by-source-node-id order**.
- It then appends **one** staged output temp path, then any `args`:

- The script reads **ONLY** the supplied inputs and writes **ONLY** the supplied output (Docs Chain owns the atomic rename to the live artefact, §8).

Run it through Docs Chain:

```
docs_chain sync    .docs_chain/contexts/sql_definitions.yaml
docs_chain verify .docs_chain/contexts/sql_definitions.yaml
```

Direct manual invocation (for evidence / debugging only) mirrors the staging: copy each DDL file into a temp content file in node-id order, then:

```
sql_to_doc.sh <staged_in_1> ... <staged_in_N> docs/database/
                SQL_REFERENCE.md
```

Optional input-count pin

If a future context adds trailing `args:`, set `SQL_TO_DOC_NUM_INPUTS=<N>` so the script knows exactly how many leading positionals are inputs (the positional at index `N+1` is then the output path). With no env var the script treats all positionals except the last as inputs.

Output

A deterministic (§11.4.50 byte-stable) Markdown document:

1. A header naming the authority direction + a “GENERATED FILE” warning.
2. A global **Table Inventory** — every distinct `CREATE TABLE` name across all inputs, alphabetically sorted, deduplicated, with a total count.
3. Per-input **Block NN** sections (numbered deterministically in Docs Chain pass order) listing that block’s tables (the original file path is *not* recoverable from a staged temp file by contract, so the zero-padded block index is the stable identifier).

Parsing recognises `CREATE TABLE name`, `CREATE TABLE IF NOT EXISTS name`, optional back-tick / double-quote / bracket quoting, case-insensitively.

Edge cases

- **Empty DDL block** (no `CREATE TABLE`): emits a “*No CREATE TABLE statements in this block.*” note; the block still appears.
- **All inputs empty**: the global inventory emits “*No CREATE TABLE statements found in any DDL source.*” and exits 0.
- **Malformed / binary input**: never crashes; exits 0 with whatever (zero or more) valid table names it could extract.
- **Too few args** (`< 2`): exits **64** with a usage message on stderr (a real FAIL, never a silent pass — §11.4.1).
- **Byte-stability**: no timestamps, host data, or temp-file names ever appear in the output, so identical input always yields byte-identical output (the §11.4.50 requirement the Docs Chain Phase-2 meta-test enforces).

Internal behaviour

The script builds the document into in-temp-dir accumulators (sibling temps in the output directory so the move stays on one filesystem), then re-assembles header → global inventory → per-block sections in a single deterministic awk splice, and mv's the result onto the Docs-Chain-supplied output path. All accumulator temps are cleaned on every exit path (`trap ... EXIT INT TERM`, §11.4.14).

Tests

`scripts/dc/test_sql_to_doc.sh` — 100% branch coverage (§11.4.27), anti-bluff (§11.4): multi-input happy path, byte-stability diff across two runs (§11.4.50), single-input, all-empty, malformed, argument-error (exit 64), the `SQL_TO_DOC_NUM_INPUTS` env-pin path, and a real-Core-DDL end-to-end case (SKIP-with-reason per §11.4.3 if the core submodule tree is absent — never a faked PASS). Captured result: **PASS=29 FAIL=0**.

```
sh scripts/dc/test_sql_to_doc.sh
```

Related scripts

- `scripts/export_markdown.sh` — generates the `.html` + `.pdf` siblings of `SQL_REFERENCE.md` (§11.4.65).
- The Docs Chain engine (`docs_chain/`) — the universal sync engine that drives this transform.

Last verified date

2026-06-17 — `sh -n parse OK`; `sh scripts/dc/test_sql_to_doc.sh` → **PASS=29 FAIL=0**; full 21-file run byte-identical across two invocations (162 distinct tables).